

Manual de Arquitectura

PharmaMobil

Clean Architecture + MVVM + Koin — Sesión 5, Reto 02 (GAA)

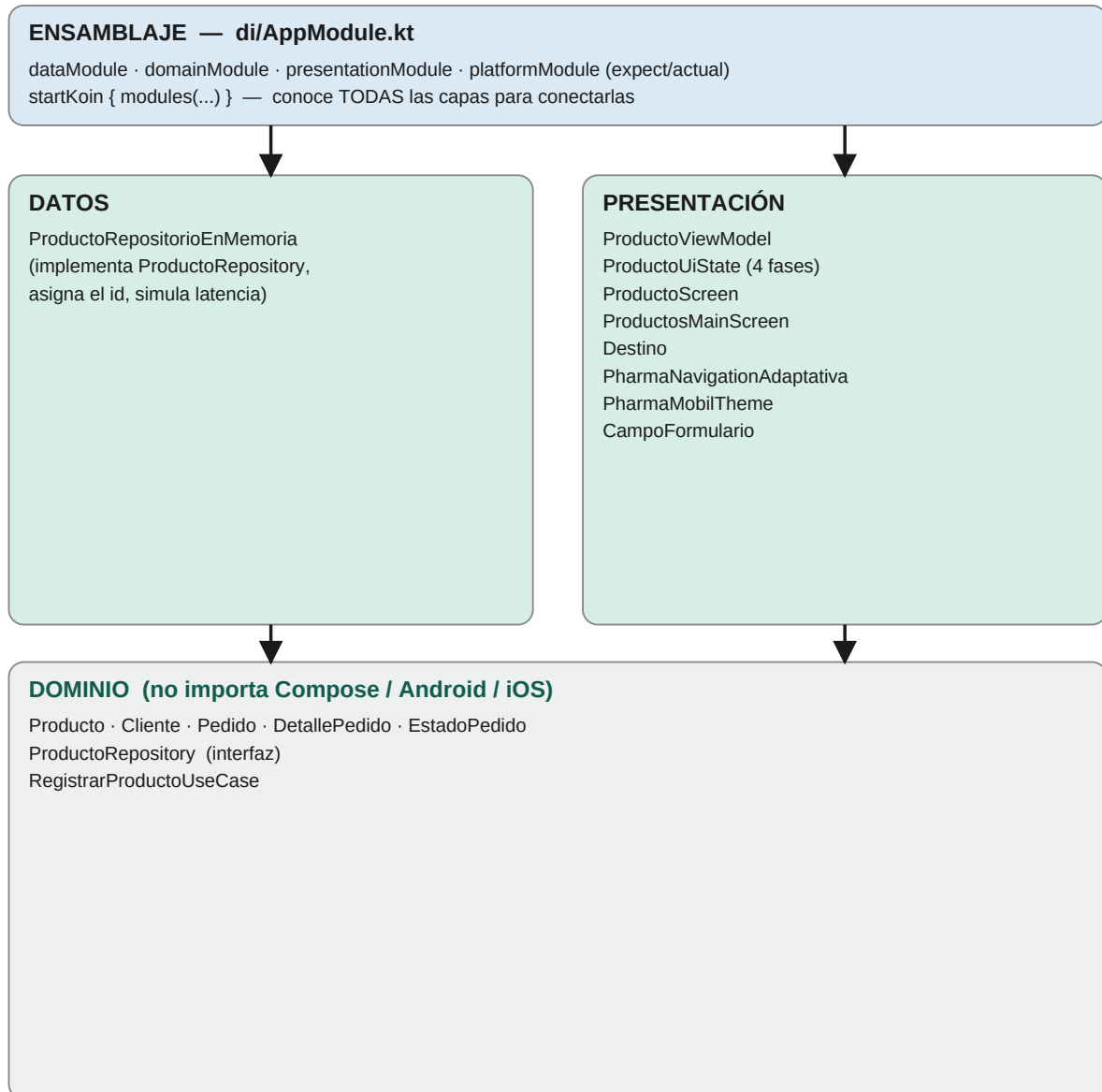
Autor: Jeanpierre Samuel Miranda Gamarra
Repositorio: github.com/JeanpierreSam/PharmaMobil
Rama: feature/clean-mvvm

Índice

1. Mapa de capas
2. Tabla de responsabilidades
3. Decisiones justificadas
4. Pruebas de flujo
5. Bitácora

1. Mapa de capas

Diagrama de las tres capas de Clean Architecture sobre el módulo de Productos, con las clases reales del proyecto. Ninguna flecha sale del dominio hacia afuera: Datos y Presentación conocen al Dominio, el Dominio no conoce a nadie. Elaboración propia.



2. Tabla de responsabilidades

Una fila por clase relevante del módulo de Productos, incluyendo clases heredadas de sesiones anteriores (S1-S4).

Clase	Capa	Responsabilidad	Prohibido	Se rompería...
Producto	Dominio	Modela el producto farmacéutico y sus reglas propias (stock, precio, reposición).	Importar Compose/Android/iOS; conocer el repositorio.	Todo el sistema pierde su unidad de negocio central; nada compilaría.
Cliente	Dominio	Modela un cliente y resuelve el teléfono ausente con un valor por defecto.	Depender de la UI o de un repositorio.	CientesScreen y DominioModeloTest dejan de compilar.
Pedido	Dominio	Agrega detalles de pedido; calcula total, cantidad de ítems y describe su estado.	Formatear moneda o texto de UI directamente.	Se pierde el cálculo del total y la descripción exhaustiva por EstadoPedido.
DetallePedido	Dominio	Representa una línea de pedido (producto + cantidad) y calcula su subtotal.	Aceptar cantidades <= 0 (lo bloquea su propio init).	Pedido.total() no tendría de dónde sumar subtotales.
EstadoPedido	Dominio	sealed class que enumera los estados posibles de un pedido.	Añadir estados fuera de la jerarquía sellada.	El when exhaustivo de Pedido.descripcionEstado() dejaría de compilar.
ProductoRepository (interfaz)	Dominio	Declara el contrato de acceso a productos (registrar, listar) sin decir cómo.	Contener lógica, delay() o detalles de almacenamiento.	El dominio y el ViewModel no tendrían con quién hablar; Koin no sabría qué inyectar.
RegistrarProductoUseCase	Dominio	Valida los datos crudos del formulario y orquesta el registro vía el repositorio.	Tocar Compose, formatear mensajes de UI, conocer ProductoRepositoryEnMemoria.	El ViewModel tendría que revalidar por su cuenta; se duplicaría la regla de negocio.
ProductoRepositoryEnMemoria	Datos	Implementa ProductoRepository en memoria: guarda la lista, asigna el id, simula latencia.	Ser referenciada directamente por el ViewModel o el caso de uso.	Koin no tendría una implementación real que inyectar en dataModule.
ProductoViewModel	Presentación	Guarda el ProductoUiState, llama al caso de uso/repositorio y expone un StateFlow de solo lectura.	Contener vistas de Compose o construir un Producto a mano.	Las pantallas de Productos no tendrían estado que observar.
ProductoUiState	Presentación	Modela las 4 fases de la pantalla de Productos y el estado de guardado/mensaje.	Contener lógica de negocio o llamadas suspendidas.	El when exhaustivo de ProductosMainScreen no tendría qué pintar.
ProductoScreen	Presentación	Formulario de registro: captura texto, resalta errores en vivo y llama a viewModel.registrar.	Validar reglas de negocio o construir el dominio.	Se pierde el formulario de alta de productos.
ProductosMainScreen	Presentación	Pinta los tabs (Activos/Inactivos/Bajo Stock) y las 4 fases; aloja el diálogo de registro.	Calcular el siguiente id o filtrar sin usar domain/query.	La pantalla principal de inventario desaparece.
Destino	Presentación	sealed class que enumera los 4 destinos de navegación (ruta, título, ícono).	Acoplarse a una pantalla concreta.	El Drawer/Rail no tendría qué listar ni cómo resaltar la selección.
PharmaNavigationAdaptativa	Presentación	Elige Drawer modal, Rail o Drawer permanente según el ancho disponible.	Contener reglas de negocio de Productos/Clientes/Pedidos.	Se pierde la navegación adaptativa y el Scaffold/Snackbar centralizados.
PharmaMobilTheme	Presentación	Define el ColorScheme claro/oscuro de Material 3 para toda la app.	Depender de una pantalla específica.	Cada pantalla definiría sus propios colores; se rompe la consistencia visual.
CampoFormulario	Presentación	Campo de texto reutilizable (state hoisting: valor y error llegan desde arriba).	Decidir cuándo mostrarse en rojo (lo decide quien lo usa).	Cada formulario reimplementaría su propio OutlinedTextField.
AppModule.kt (data/domain/presentation/platform Module)	Ensamblaje	Declara qué implementación concreta corresponde a cada interfaz/caso de uso/ViewModel para Koin.	Contener lógica de negocio o de UI.	Nadie tendría sus dependencias resueltas; koinViewModel() fallaría en tiempo de ejecución.

3. Decisiones justificadas

a) ProductoUiState: sealed interface Fase + data class, vs. banderas booleanas

Con un único estado sellado, el compilador obliga a cubrir las cuatro fases (Cargando, SinProductos, ConProductos, Error) en cada when. Con banderas sueltas (cargando: Boolean, error: String?, productos: List<Producto> todas al mismo nivel) existirían combinaciones que el tipo permite pero que no tienen sentido de negocio -- por ejemplo cargando = true junto con un error no nulo, o una lista con datos y un mensaje de error simultáneos -- y esas combinaciones habría que descartarlas a mano en cada pantalla, sin que el compilador avise si alguien olvida un caso. El PedidoViewModel del material de la sesión es justo ese ejemplo defectuoso: banderas independientes que permiten estados imposibles.

b) Traslado de ValidacionProducto.kt al caso de uso

Antes, validarYCrearProducto vivía en presentation/producto/ y construía el Producto directamente: la regla de negocio quedaba atrapada en la capa de UI, así que cualquier otro punto de entrada (una API, una carga masiva, una prueba) tendría que reimplementarla. Al moverla a RegistrarProductoUseCase (domain/usecase) la regla es reutilizable, independiente de Compose y testeable sin arrancar la UI. El proyecto todavía no tiene un ClienteValidator -- ClientesScreen es solo una vista de demostración sin formulario -- pero cuando se implemente el alta de Clientes, la intención es aplicar el mismo patrón desde el inicio (un caso de uso en domain/usecase), sin pasar primero por un validador en presentación.

c) Cambiar ProductoRepositorioEnMemoria por un repositorio remoto

Una sola línea, dentro de dataModule en di/AppModule.kt: `single<ProductoRepository> { ProductoRepositorioEnMemoria() }` pasaría a `single<ProductoRepository> { ProductoRepositoryRemoto(get()) }`. Ni ProductoViewModel, ni RegistrarProductoUseCase, ni ProductoScreen necesitarían tocarse, porque todos dependen de la interfaz ProductoRepository, no de la implementación concreta -- es la Inversión de Dependencias funcionando en la práctica.

4. Pruebas de flujo

En `shared/src/commonTest/.../presentation/producto/ProductoViewModelTest.kt` se cubren las 4 fases del ViewModel con repositorios falsos, sin emulador ni interfaz gráfica:

- `repositorioVacio_produceSinProductos`
- `repositorioConProductos_produceConProductos`
- `repositorioQueFalla_produceError`
- `precioCero_dejaMensajeDeErrorSinLlamarAlRepositorio`

Nota técnica: `viewModelScope` despacha en `Dispatchers.Main`, que no existe en un test JVM/Native puro. Se agregó `Dispatchers.setMain(UnconfinedTestDispatcher())` en `@BeforeTest` (y `resetMain()` en `@AfterTest`) para que las corrutinas del ViewModel se ejecuten de forma inmediata dentro de `runTest`; sin este cambio las 4 pruebas quedaban atascadas en `Fase.Cargando` (ver Bitácora).

4.1 Matriz de pruebas de flujo

#	Condición inicial forzada	Acción ejecutada	Estado esperado	Estado observado	Resultado
1	Repositorio falso vacío	<code>cargarProductos()</code>	<code>Fase.SinProductos</code>	<code>Fase.SinProductos</code>	Correcto
2	Repositorio falso con productos	<code>cargarProductos()</code>	<code>Fase.ConProductos</code>	<code>Fase.ConProductos</code>	Correcto
3	Repositorio falso que lanza excepción	<code>cargarProductos()</code>	<code>Fase.Error</code>	<code>Fase.Error</code>	Correcto
4	Formulario con precio "0"	<code>registrar(...)</code>	Mensaje de error, sin llamada al repositorio	<code>mensaje = "El precio debe ser mayor que cero."</code>	Correcto

4.2 Ejecución

Android (`androidHostTest`, JVM): `./gradlew :shared:testAndroidHostTest -- 35/35` pruebas exitosas, incluidas las 4 de esta tabla (ver evidencias-s5.pdf, sección 2).

iOS (`iosSimulatorArm64Test`): el código de prueba **compila** correctamente para el target iOS (`compileTestKotlinIosSimulatorArm64` exitoso), pero la ejecución del simulador requiere macOS y no se pudo correr desde este entorno Windows. Se declara honestamente como limitación de la sesión, pendiente de verificar en una Mac.

5. Bitácora

Dificultades reales encontradas durante el Reto 01 y su resolución.

1. El id del producto lo calculaba la UI, no el repositorio

Problema: Antes de este Reto, ProductosMainScreen calculaba "(productos.maxOrNull { it.id } ?: 0L) + 1L" y ProductoScreen llevaba su propio contador local — dos fuentes de verdad para el mismo id.

Solución: Se centralizó la asignación del id dentro de ProductoRepositorioEnMemoria.registrar(), y se eliminó todo el cálculo de id en la capa de presentación.

2. AppModule.kt no compilaba: "Unresolved reference 'includes'"

Problema: La construcción inicial de initKoin() usaba "config?.let { includes(it) }" para aplicar la configuración de plataforma (androidContext/androidLogger), pero includes() no es la función de Koin para ejecutar un KoinAppDeclaration.

Solución: Se corrigió a "config?.invoke(this)", invocando el lambda directamente sobre el receptor KoinApplication del bloque startKoin { }.

3. Las 4 pruebas de ProductoViewModelTest quedaban en Fase.Cargando

Problema: viewModelScope.launch despacha en Dispatchers.Main, que no está instalado en un test JVM/Native puro; la corrutina del ViewModel nunca llegaba a ejecutarse dentro de runTest.

Solución: Se agregó Dispatchers.setMain(UnconfinedTestDispatcher()) en @BeforeTest y Dispatchers.resetMain() en @AfterTest.

4. androidApp no compilaba tras agregar Koin: "Unresolved reference 'koin'" en MainApplication.kt

Problema: koin-android estaba declarado como implementation(...) dentro del source set androidMain de shared, así que no se exponía a androidApp, que es quien realmente llama a androidContext()/androidLogger().

Solución: Se cambió a api(libs.koin.android) en shared/build.gradle.kts para propagar la dependencia al módulo consumidor.

5. No se lograba capturar la Fase.Cargando en el emulador

Problema: ProductoViewModel es compartido (misma ViewModelStoreOwner) y se crea apenas se compone Inicio -- el destino inicial de la app --, así que para cuando se navegaba manualmente al Drawer y a Productos, la carga ya había terminado incluso con el delay original.

Solución: En una rama temporal de evidencia se abrió la app directo en Productos y se alargó el delay del repositorio a ~9 s, dando margen para la captura; los cambios se revirtieron después (no forman parte de feature/clean-mvvm).

6. Los commits del Reto 01 y el Reto 02 quedaron mezclados en uno solo

Problema: El primer commit incluyó, además del refactor de capas (GP), el archivo ProductoViewModelTest.kt que corresponde a la Sección 4 del Reto 02 (GAA).

Solución: Se deshizo el commit con git reset --soft HEAD~1 y se volvió a comitear en dos pasos: primero los archivos del Reto 01, después ProductoViewModelTest.kt por separado.